Solving the Dog Game

Live demo: charith-reddy-pareddy.github.io/dog-game

Code: github.com/Charith-Reddy-Pareddy/dog-game, reproducible with `pytest` and the commands in Section 8.

This report studies the dog game, a two-player, general-sum game with continuous actions, with two families of solver: policy-gradient self-play (REINFORCE, A2C and PPO) and Nash-Q on a discretized grid. It answers the research questions in RESEARCH_QUESTIONS.md from saved runs, and says how many runs each claim rests on.

Summary

- **Policy-gradient self-play converges on the dog game.** 74 of 75 REINFORCE runs and 25 of 25 PPO runs reached exploitability below 0.004 within 1500 episodes. PPO needs far fewer episodes: 19 of 25 runs had converged after 100, against none for REINFORCE and A2C (Section 5.2), an advantage that comes from reusing each episode (with one epoch PPO is no faster than REINFORCE), and on the walking game PPO is the least stable of the three (Sections 5.10 and 5.11).
- **Self-play never finds the solver's interior equilibrium when the houses share a coordinate.** All 30 runs of the first study ended on a boundary equilibrium, one of two mirror images (23 to 7). With 50 seeds per noise level, which mirror image wins is consistent with a fair coin (31 to 19 at the default noise, p = 0.12), so the earlier 3-to-1 split was a small-sample effect (Sections 5.3 and 5.19).
- **Dog inertia slows training sharply.** On the harder setup 4, 5, 1 and 0 of 5 runs converged at dog weights 0, 0.3, 0.6 and 0.9 (Section 5.4).
- **Pre-training helps or hurts depending on the source.** From a setup with the same equilibrium corners, every run was under the threshold after 100 episodes; from one with flipped corners, every run was worse than starting from scratch, mainly because pre-training collapses the exploration noise: resetting it removes most of the harm (Section 5.5).
- **Nash-Q on a grid.** The stage-game error falls about as 1 over the fictitious-play iterations, and re-solving less often changes nothing in this game. The grid is exact from 5 x 5, the exact equilibrium is always a single action pair, and all of fictitious play's apparent mixing is noise (Sections 5.6 to 5.8).
- **Walking game.** REINFORCE and the 10-direction Nash-Q network both end near the one-shot Nash corners in all 5 seeds (Section 5.9). With A2C and PPO added, the ranking changes: A2C is the most reliable (5 of 5 seeds at the corners), while PPO is the fastest to get there but swings away again in 3 of 5 seeds. A smaller learning rate does not fix it. One epoch does, but only given three times more training, and default PPO still has two seeds stuck far from the corners after 900 episodes (Sections 5.10 to 5.12). A lower discount makes the walking game easier to learn and a higher one harder (Section 5.12).
- **Exact dynamic results.** With dog inertia, a closed form says exactly when holding the one-shot Nash actions is a feedback (Markov-perfect) equilibrium, and an explicit piecewise-affine policy is a feedback equilibrium whenever the houses lie on either side of the dog; both match the dynamic program to one grid step (Sections 5.15 and 5.20). The one-shot game is an exact potential game, flat along its continuum of equilibria; there, learning drifts along the flat direction until the box stops it, and the dynamic game is not a Markov potential game once policies react to the dog (Section 5.19).
- **Open:** the feedback equilibrium when a one-shot action is interior, the equilibrium of the walking game, continuous-time q-learning, and whether the findings transfer to harder games (Section 6).

1. Background

The dog game (also called the Battling Influencers Game) is a small, general-sum, two-player game with continuous actions. It is useful as a proving ground for multi-agent learning methods because it has a closed-form equilibrium to check against: any learned policy can be scored exactly, so "did it converge?" has a number for an answer instead of an opinion.

Two families of solver are studied here, side by side on the same environment:

- **Policy gradient**: self-play with REINFORCE, A2C and PPO, using a Gaussian policy over continuous positions.
- **Nash-Q**: a deep-Q style method on a discretized action grid, where every Bellman target requires solving a stage game for its Nash equilibrium, either approximately (fictitious play) or exactly (Lemke-Howson).

The report answers the research questions in RESEARCH_QUESTIONS.md from experiments that can be rerun from this repository. Every number below comes from a saved result file in `results/`, and every claim says how many runs it rests on.

2. The game

Two players, red and blue, each own a house at a fixed point in the unit square. Each round both players simultaneously pick a point in the square, and a shared dog moves to the weighted average `w * red + (1 - w) * blue` of the two picks. Each player is rewarded by the negative squared distance from the dog to their own house, and the discount is 0.9. The game is general-sum, not zero-sum. It has the same structure as two influencers each staking out an opinion to pull a shared audience toward them. Because the domain is bounded, a player who wants the dog exactly on their house has to overshoot: pick further out than their own house to counteract the other player's pull.

2.1 Why the equilibrium sits on the boundary

Substituting each player's unconstrained best response into the other's shows there is no interior fixed point unless the houses share a coordinate. The unconstrained recursion pushes the actions further out on every iteration. The bounded domain turns this into a genuine equilibrium: both players are pushed to the boundary and exaggerate as much as the square allows. Iterated best response (`doggame/nash.py`) reaches this fixed point in two rounds, and it is used throughout as the ground truth.

2.2 Uniqueness

The equilibrium is unique when the two houses differ in every coordinate. Each coordinate the houses share becomes a degenerate direction: any pair of actions on that axis that averages back to the shared value is equally good, so the equilibrium set is a continuum. Identical houses are the extreme case, where no exaggeration is needed at all.

2.3 Two variants

- **Dog inertia.** `make_inertial_transition(w0)` gives the dog momentum: `dog' = w0 * dog + (1 - w0) * (w * red + (1 - w) * blue)`. Holding both actions fixed, the dog still ends at the same steady state as without inertia; it only gets there more slowly, at rate `w0` per round.
- **The walking game (the angle-radius version).** In the main game a player jumps straight to any point it likes. In the walking game each player has a position, starting on its own house, and each round picks a direction (an angle) and a step length of up to 0.2, then walks that way from where it stands, stopping at the edge of the square. The dog moves to the average of the two players' new positions, and each player is rewarded by the negative squared distance from the dog to its own house, as before. The state is

therefore the dog's position plus both players' positions, and an action is a polar pair (angle, step length) instead of a point. The angle comes from a von Mises distribution and the step fraction from a Beta distribution, so the angle wraps cleanly and nothing is clipped. This is one reading of "angle-radius action", described in ASSUMPTIONS.md.

3. Methods

3.1 Policy gradient

A Gaussian policy (`doggame/agents.py`) gives both players' action means through one of three network layouts, plus a learned per-player log standard deviation (initialized at -0.5, a standard deviation of about 0.61). Training is self-play, one episode of 20 rounds per update, with Adam at learning rate 0.01 through PyTorch's optimizer. Three algorithms share this setup:

- **REINFORCE** (`doggame/train.py`): the log-probability of each action weighted by the discounted return minus the episode's mean return.
- **A2C** (`doggame/train_ac.py`): the same, but the baseline is a learned value function of the dog's position, one per player.
- **PPO** (`doggame/train_ac.py`): A2C with each episode reused for 4 epochs and the probability ratio clipped at 0.2.

Three network layouts sit behind one interface (`doggame/networks.py`): **separate** (two independent networks), **shared** (one trunk with two heads) and **partial** (a shared trunk feeding two separate per-player branches).

3.2 Nash-Q on a discretized grid

`doggame/discretize.py` turns the square into an N x N grid of actions and builds both players' payoff matrices with the same transition and reward functions as the continuous game. `doggame/dqn.py` iterates the Nash-Q backup `Q = R + discount * NashV(Q)` to a fixed point. Because the dog game's transition does not depend on the incoming state, this collapses to iterating on one pair of payoff matrices, and the fixed point has a closed form: the discounted value equals the one-shot Nash value divided by `1 - discount`. That shortcut is specific to this game. The stage game is solved either by fictitious play (`doggame/fictitious_play.py`, approximate) or exactly by NashPy's Lemke-Howson algorithm (`doggame/exact_nash.py`). The exact solver is hardened: every answer is checked to be an equilibrium, retried from another starting label if not, and cut off after a fixed number of pivots, so a result never depends on how fast the machine is.

3.3 The walking game

`doggame/polar_agent.py` trains the walking game's policy with REINFORCE, A2C or PPO (the actor-critic loop of `doggame/train_ac.py` is shared with the main game). `doggame/polar_dqn.py` is a deep Nash-Q for the same game: 10 compass directions, one network giving both players' 10 x 10 Q-values, a target network, a replay buffer and an exact Nash solve at every target.

4. Verification

A solved or learned policy is checked directly instead of trusting how it was produced (`doggame/verify.py`):

- **Exploitability**: for a continuous action pair, search a 101 x 101 grid for the largest gain either player could get by deviating. It is 0 at an equilibrium.
- **Indifference gap**: for a discrete mixed strategy, every action played with positive probability should have equal value, and no unplayed action should do better. The check uses a probability floor (`tol`, 0.001) to

decide what counts as played.

A defect found and fixed. On a fine enough action grid, a uniform mixed strategy spreads every probability below the floor, so the check selected no actions and silently returned infinity instead of a number. It now falls back to the most probable actions when none clear the floor. This is covered by a regression test, and it matters for the resolution question below.

5. Experiments

Every experiment is a function in [`doggame/dog_game_studies.py`](doggame/dog_game_studies.py) or [`doggame/dog_game_study.py`](doggame/dog_game_study.py). Raw results are in `results/dog_game_runs.jsonl` and `results/dog_game_studies.jsonl`, one line per run, and the tables below are produced from them by `doggame/dog_game_summary.py`. A run counts as **converged** when its exploitability is below 0.004. That threshold is my choice, so each table also shows the median.

5.1 Does policy-gradient self-play converge?

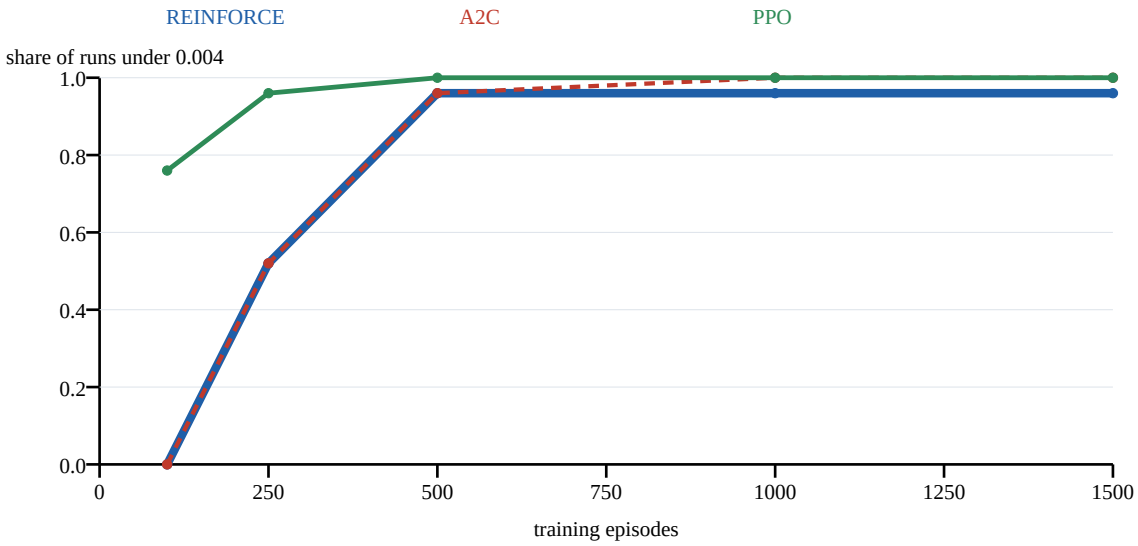
REINFORCE self-play, 1500 episodes, five house setups and three network layouts, five seeds each (75 runs). Each cell is the number of runs under 0.004 with the median exploitability.

House setup	separate	shared	partial
(0.9, 0.2) / (0.1, 0.8), w = 0.5	5 of 5 (median 0.0002)	5 of 5 (median 0.0002)	5 of 5 (median 0.0004)
(0.8, 0.8) / (0.2, 0.3), w = 0.3	4 of 5 (median 0.0007)	5 of 5 (median 0.0006)	5 of 5 (median 0.0008)
(0.7, 0.1) / (0.6, 0.9), w = 0.7	5 of 5 (median 0.0003)	5 of 5 (median 0.0003)	5 of 5 (median 0.0008)
(0.95, 0.5) / (0.05, 0.5), w = 0.5	5 of 5 (median 0.0001)	5 of 5 (median 0.0001)	5 of 5 (median 0.0002)
(0.6, 0.6) / (0.4, 0.4), w = 0.5	5 of 5 (median 0.0001)	5 of 5 (median 0.0001)	5 of 5 (median 0.0002)

- 74 of 75 runs converged. The one miss is the `separate` layout on setup 2, seed 0, at 0.18. The other four seeds of that setup converge. An earlier single-seed note said that training the same run 3000 episodes longer brought it to 0.004; that was not rerun here.
- The three layouts do not separate: `separate` 24 of 25, `shared` 25 of 25, `partial` 25 of 25. The one miss belongs to `separate`, but a single run cannot rank architectures.
- The threshold matters: the closest passing run is at 0.0039 (`shared`, setup 2, seed 0), so the count depends on where the line sits.

5.2 REINFORCE, A2C and PPO

The same five setups and seeds, the `separate` layout, 1500 episodes, scored at five points during training (25 runs per algorithm). The chart shows the share of runs under 0.004 by episodes trained.



Algorithm	100 episodes	250 episodes	500 episodes	1000 episodes	1500 episodes	Median at the end
REINFORCE	0 of 25	13 of 25	24 of 25	24 of 25	24 of 25	0.0002
A2C	0 of 25	13 of 25	24 of 25	25 of 25	25 of 25	0.0001
PPO	19 of 25	24 of 25	25 of 25	25 of 25	25 of 25	0.0000

- **PPO is much faster per episode.** 19 of 25 PPO runs converged after 100 episodes, against none for REINFORCE and A2C, and all 25 converged by 500.
- **A2C is not faster than REINFORCE** for the first 500 episodes, but it converged on all 25 runs by 1000 episodes, including the slow run that REINFORCE left unconverged.
- **The comparison is matched on episodes, not on gradient steps.** PPO takes four gradient steps on each episode and REINFORCE takes one. Section 5.11 reruns PPO with fewer epochs to separate the two.

5.3 Which equilibrium does self-play find?

In the setup whose houses share a y-coordinate, the equilibrium set has a continuum of members. Iterated best response (`doggame/nash.py`) returns the interior point with both players at y = 0.5. Thirty REINFORCE runs (ten seeds per layout) ended here:

Layout	red top, blue bottom	red bottom, blue top	both at y = 0.5	other	Median exploitability
separate	8 of 10	2 of 10	0 of 10	0 of 10	0.00009
shared	7 of 10	3 of 10	0 of 10	0 of 10	0.00008
partial	8 of 10	2 of 10	0 of 10	0 of 10	0.00018

- **Never the interior point.** All 30 runs ended with one player at the top edge and the other at the bottom edge. Both are exact equilibria (exploitability below 0.0002), as is the interior point.
- **Two mirror equilibria, 23 to 7 here.** 23 runs ended with red at the top and 7 with red at the bottom, with the layouts alike (8, 7 and 8 of 10). The layouts share seeds 0-9, so these are not 30 independent draws; with 50 seeds the split is 31 to 19, consistent with a fair coin (Section 5.19).

- **The exact solver agrees with the learners.** Lemke-Howson on the discretized game also returns a boundary equilibrium for this setup, at every grid size (Section 5.7).
- **Why.** Section 5.19 tests this with 50 seeds per noise level and two ablations: learning random-walks along the flat direction until the box stops it.

5.4 Does dog inertia change the outcome?

REINFORCE, five seeds, 1500 episodes, with the dog's own weight `w0` set to 0, 0.3, 0.6 or 0.9, on the easiest setup and on the setup that was slowest in Section 5.1. Policies are scored against the one-shot game.

House setup	w0 = 0.0	w0 = 0.3	w0 = 0.6	w0 = 0.9
(0.9, 0.2) / (0.1, 0.8), w = 0.5	5 of 5 (median 0.0002)	5 of 5 (median 0.0002)	5 of 5 (median 0.0002)	5 of 5 (median 0.0006)
(0.8, 0.8) / (0.2, 0.3), w = 0.3	4 of 5 (median 0.0007)	5 of 5 (median 0.0016)	1 of 5 (median 0.0102)	0 of 5 (median 0.0295)

- **Inertia slows training a great deal on the hard setup.** 4, 5, 1 and 0 of 5 runs converged within 1500 episodes at `w0` = 0, 0.3, 0.6 and 0.9. On the easy setup every run converged at every `w0`, with a slightly higher median at 0.9.
- **The unconverged runs look unfinished, not different.** Their median distance to the one-shot equilibrium stays small (0.0016, 0.0011, 0.0072 and 0.0176 over all runs at each `w0`), but the worst runs at `w0` of 0.6 and 0.9 are about 1.0 away, meaning one player is still far from its corner. Whether more episodes fix them was not tested.
- **What this does not say.** Policies are scored against the one-shot equilibrium, which is the steady state of the dynamic game. Whether the equilibrium of the full game with inertia is the same is not solved here.

5.5 Does pre-training help?

REINFORCE on setup 2, the slowest one, five seeds. "Pre-trained" means 1500 self-play episodes on another setup first, then training on setup 2. Equilibrium corners from the exact solver: setup 1 has red at bottom-right and blue at top-left; setups 2 and 5 have red at top-right and blue at bottom-left.

Starting point	after 100 episodes	after 300 episodes
from scratch	0 of 5 (median 0.0599)	2 of 5 (median 0.0048)
pre-trained on setup 1	0 of 5 (median 0.1599)	0 of 5 (median 0.2289)
pre-trained on setup 5	5 of 5 (median 0.0002)	4 of 5 (median 0.0002)

- **It depends on the source.** Pre-training on setup 5, which has the same corners as setup 2, put all 5 runs under 0.004 after 100 episodes, where training from scratch had none.
- **It can hurt.** Pre-training on setup 1, whose corners are flipped, left every run worse than scratch after 300 episodes (median 0.23 against 0.0048).
- **Why: pre-training collapses the exploration noise.** A follow-up (10 seeds) pre-trains on setup 1 and then resets the policy's exploration noise to its initial value, de-saturates its sigmoid (scales the output layer so the largest pre-sigmoid output is 2, still pointing at the same corners), or both, before training on setup 2:

Start	Under 0.004 after 100	after 300	Median exploitability after 300	After pre-training: largest pre-sigmoid output, mean log-std
scratch	0 of 10	5 of 10	0.0043	-
pretrained	0 of 10	0 of 10	0.2248	8.5, -2.81
reset noise	0 of 10	3 of 10	0.0078	8.5, -2.81
desaturated	0 of 10	0 of 10	0.1938	8.5, -2.81
both	0 of 10	6 of 10	0.0036	8.5, -2.81

After pre-training the noise has shrunk to a standard deviation of about 0.06 (mean log-std -2.81) and the sigmoid is saturated (pre-sigmoid outputs up to 8.5). Resetting the noise removes most of the harm (median 0.0078 against 0.22) and de-saturating alone does not (0.19); with both, the runs do as well as from scratch (6 of 10 against 5 of 10 under the threshold). So the harm comes mainly from collapsed exploration, not from having to un-learn a direction; de-saturating also moves the actions part of the way to the center, so it is not a pure test of saturation.

5.6 How precisely must the stage game be solved?

Nash-Q on the 9 x 9 grid of setup 1, discount 0.9. Fictitious play replaces the exact solver with a given number of iterations, and the stage game is re-solved every 1, 5 or 25 outer iterations. Each cell is the median error of the fixed point's Q-values against the exact solution, and the median gap (the most either player could gain by switching to a single action) of the final strategy. Five seeds per cell.

Fictitious-play iterations	re-solve every 1: Q error / gap	re-solve every 5: Q error / gap	re-solve every 25: Q error / gap
10	0.3612 / 0.0583	0.3612 / 0.0583	0.3612 / 0.0583
30	0.1311 / 0.0215	0.1311 / 0.0215	0.1311 / 0.0215
100	0.0406 / 0.0067	0.0406 / 0.0067	0.0406 / 0.0067
300	0.0136 / 0.0023	0.0136 / 0.0023	0.0136 / 0.0023
1000	0.0041 / 0.0007	0.0041 / 0.0007	0.0041 / 0.0007
3000	0.0014 / 0.0002	0.0014 / 0.0002	0.0014 / 0.0002

- **The error falls about as 1 over the number of iterations.** It is 0.36 at 10 iterations and 0.0014 at 3000, so a given Q error can be bought with a predictable number of iterations.
- **Re-solving less often changes nothing here.** Every column is the same. In this game the transition does not depend on the state, so each Q matrix is the payoff matrix plus a constant, and adding a constant never moves an equilibrium. This will not hold in a game where the continuation value varies by state.

5.7 Does grid resolution matter?

The exact (Lemke-Howson) solution of the discretized stage game on five grid sizes, for all five setups (25 games).

Grid	Actions per player	Matrix entries	Setups with a unique equilibrium found (of 4)	Shared-y setup: distance to the solver's equilibrium	Equilibrium is one action pair	Seconds (median)
3 x 3	9	81	2	1.00	yes	0.003
5 x 5	25	625	4	1.00	yes	0.031
9 x 9	81	6,561	4	1.00	yes	0.356

Grid	Actions per player	Matrix entries	Setups with a unique equilibrium found (of 4)	Shared-y setup: distance to the solver's equilibrium	Equilibrium is one action pair	Seconds (median)
13 x 13	169	28,561	4	1.00	yes	2.017
17 x 17	289	83,521	4	1.00	yes	116.625

- **Quality stops improving at 5 x 5.** The equilibrium sits on the boundary, and every grid contains the corners, so a finer grid finds nothing new. A 3 x 3 grid is too coarse: for some setups it picks a different corner.
- **Cost grows steeply.** The payoff matrices have the square of the number of actions as entries, and the exact solve time grows at least as fast. These timings come from a heavily loaded machine, so only the growth matters, not the absolute seconds.
- **Ties make it far slower than the median shows.** On the setup with the most tied payoffs (houses on a diagonal), the exact solve took 17 seconds at 9 x 9, 338 seconds at 13 x 13 and about 43 minutes at 17 x 17, because most starting labels end in an error and are retried from another label. The other setups took seconds at the same sizes.
- **The equilibrium is always one action pair.** In all 25 games the exact solution has support 1 x 1.
- **The shared-y setup returns a boundary equilibrium** at every size, 1.0 away from the interior point the best-response solver reports. Both are valid (Section 5.3).

5.8 How much of fictitious play's mixing is noise?

Fictitious play on the same game for a range of iteration counts, five seeds each, scored with the indifference check at its default floor of 0.001.

Fictitious-play iterations	Indifference spread	Actions counted as played	Weight on the leftover first pick
30	0.6415	2	0.03226
100	0.6496	2	0.00990
300	0.6519	2	0.00332
1000	0.0000	1	0.00100
3000	0.0000	1	0.00033
10000	0.0000	1	0.00010
exact (Lemke-Howson)	0.0000	1	0.00000

- **The exact equilibrium is one action pair, so none of the mixing is real.** What fictitious play shows is the weight of its first random pick, which decays as 1 over the number of iterations plus one.
- **The check hides it only once it falls below the floor.** At 300 iterations or fewer the leftover weight is above 0.001, so a second action counts as played and the indifference spread is about 0.65. From 1000 iterations the weight is at or below 0.001 and the spread is exactly 0.
- **A structural fix is available.** Dropping the first pick's weight from the running average would remove the noise instead of filtering it. It is not implemented.

5.9 The walking game

The angle-radius version, on one house setup (red at (0.88, 0.12), blue at (0.3, 0.65), w = 0.5, maximum step 0.2). The score is the sum of both players' distances from the one-shot Nash corners at the end of a 15-step game, averaged over 10 sampled games, over five seeds. REINFORCE trains 300 episodes and the Nash-Q network 60.

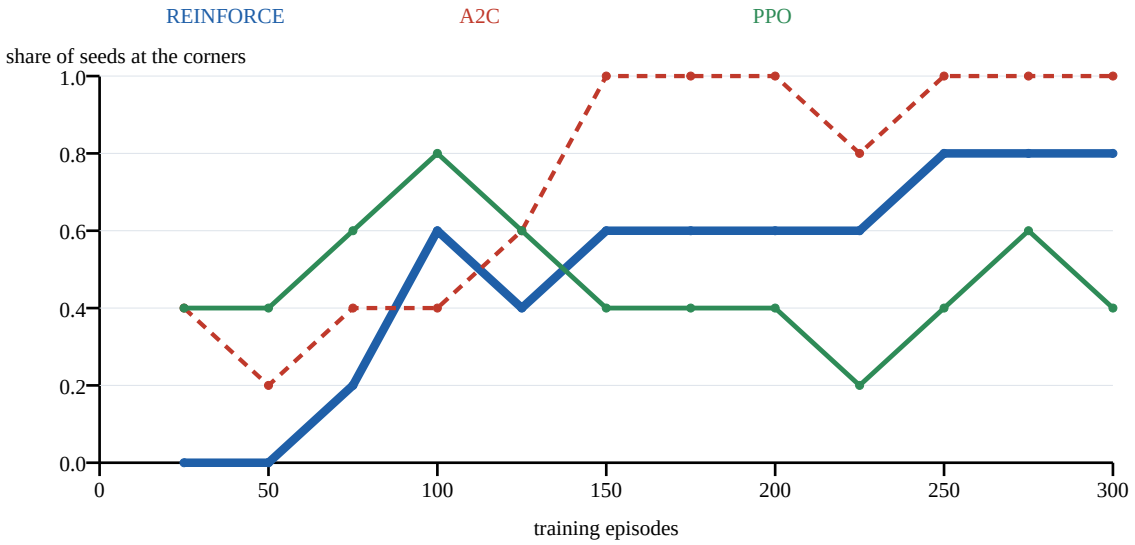
Method	Distance before training	Distance after training	Seeds that improved
REINFORCE (angle and step)	1.49	0.04	5 of 5
Nash-Q (10 directions)	1.95	0.01	5 of 5

- **Both methods walk to the corners.** The saved paths for five seeds are on the live site with a step slider and a play button.
- **What this does not show.** The one-shot corners are the equilibrium of the one-shot game. The walking game itself has not been solved, so this shows the players heading to the one-shot corners, not that those corners are an equilibrium of the walking game. One untrained REINFORCE policy (seed 4) started near the corners anyway.

5.10 REINFORCE, A2C and PPO on the walking game

A separate report, WALKING_GAME_REPORT.md, covers policy gradient on the walking game in full.

The same three algorithms on the walking game (Section 5.9's house setup), five seeds each (15 runs), 300 episodes, with the same 10-game score measured every 25 episodes. The chart shows the share of the five seeds whose summed distance from the Nash corners is below 0.1 (my threshold). In the table, "At N" is the mean distance after N episodes of training. The paths swing, so it also reports how many of the later checkpoints (from 150 episodes on, 7 per seed) are at the corners.



Algorithm	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
REINFORCE	1.49	0.696	0.341	0.130	0.055	24 of 35	4 of 5
A2C	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5
PPO	1.49	0.263	0.127	0.500	0.371	14 of 35	2 of 5

- **The ranking is not the main game's.** On the main game PPO was the fastest and most reliable. On the walking game **A2C is the most reliable**: all 5 seeds end at the corners and 34 of its 35 later checkpoints are under 0.1.
- **REINFORCE is slower and more variable.** It reaches the corners later (mean distance 0.34 at 100 episodes against 0.20 for A2C) and spends 24 of 35 later checkpoints there; one seed stays near 0.2.
- **PPO is the fastest to get there and the worst at staying.** It has the lowest distance at 50 and 100 episodes, but seeds 1, 2 and 4 leave the corners again later (their distance climbs back to about 1.0, meaning a player has left its corner), and it spends only 14 of 35 later checkpoints at the corners.
- **A single end-of-training number would mislead.** A first run of this study, scored only at 50, 100, 200 and 300 episodes, gave the same order but different per-seed outcomes (PPO seed 3 ended at 1.4 there and at 0.0 here), because evaluating the policy draws random numbers and so changes the training run. The paths are sensitive, which is why this study records every 25 episodes and reports the share of checkpoints.
- **The mechanism behind PPO instability remains unresolved.** Section 5.11 tests fewer epochs and a lower learning rate: fewer epochs improve the observed walking-game score, while the lower rate does not. Those tested settings do not identify why PPO swings away from the corners.

5.11 Where does PPO's speed come from, and can it be made stable?

Two follow-ups to Sections 5.2 and 5.10, 65 more runs.

Main game, matched on gradient steps. PPO takes one gradient step per epoch on each episode, and REINFORCE and A2C take one in total. The same five setups and seeds, the `separate` layout, with PPO at 1 and 2 epochs next to the default 4:

Algorithm	Gradient steps per episode	100 episodes	250 episodes	500 episodes	1000 episodes	1500 episodes	Median at the end
REINFORCE	1	0 of 25	13 of 25	24 of 25	24 of 25	24 of 25	0.0002
A2C	1	0 of 25	13 of 25	24 of 25	25 of 25	25 of 25	0.0001
PPO, 1 epoch	1	1 of 25	12 of 25	20 of 25	24 of 25	25 of 25	0.0002
PPO, 2 epochs	2	9 of 25	22 of 25	23 of 25	25 of 25	25 of 25	0.0000
PPO, 4 epochs (default)	4	19 of 25	24 of 25	25 of 25	25 of 25	25 of 25	0.0000

- **The speed-up comes from reusing each episode.** After 100 episodes 1, 9 and 19 of 25 runs had converged with 1, 2 and 4 epochs. With one epoch PPO is no faster than REINFORCE (20 of 25 against 24 of 25 after 500 episodes), so the clipped objective and the learned baseline are not what makes it fast.
- **Per gradient step it is not clearly ahead.** PPO with 4 epochs after 100 episodes has taken 400 steps and has 19 of 25 runs converged. REINFORCE after 250 episodes (250 steps) has 13 and after 500 (500 steps) has 24. So PPO is more efficient in samples, because each episode is used several times, but not clearly in steps.

Walking game, a gentler PPO. PPO with one epoch and/or a learning rate of 0.003 instead of 0.01, five seeds each, scored as in Section 5.10 (300 episodes):

Setting	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
A2C (reference)	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5

Setting	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
PPO, lr 0.01, 4 epochs (default)	1.49	0.263	0.127	0.500	0.371	14 of 35	2 of 5
PPO, lr 0.01, 1 epoch	1.49	0.595	0.479	0.249	0.123	21 of 35	4 of 5
PPO, lr 0.003, 4 epochs	1.49	0.477	0.361	0.448	0.362	15 of 35	1 of 5
PPO, lr 0.003, 1 epoch	1.49	0.836	0.641	0.470	0.322	8 of 35	2 of 5

- **Fewer epochs helps but does not fix it.** One epoch at learning rate 0.01 ends with 4 of 5 seeds at the corners and 21 of 35 later checkpoints there, against 2 of 5 and 14 of 35 for the default. It is slower early (mean distance 0.60 after 50 episodes, against 0.26).
- **A smaller learning rate does not help.** It gives 1 of 5 seeds at 4 epochs and 2 of 5 at one epoch. Several of those runs were still far from the corners at 300 episodes, so a longer budget might change this; that was not run.
- **None matches A2C**, which has 5 of 5 seeds and 34 of 35 later checkpoints at the corners. The instability is therefore not just a step-size problem, and reusing each episode seems to contribute, but why is not tested.
- Five seeds per setting and a 300-episode horizon, so small differences between settings should not be read as a ranking.

5.12 Longer training, and a different step length or discount, on the walking game

Training three times longer. A2C, PPO with the default 4 epochs and PPO with one epoch, five seeds each, 900 episodes, scored every 25 episodes (late checkpoints are from 450 episodes on):

Setting	Before training	At 300	At 600	At 900	Late checkpoints at the corners (from 450)	Seeds at the corners at the end
A2C	1.49	0.011	0.003	0.003	95 of 95	5 of 5
PPO, 4 epochs (default)	1.49	0.371	0.499	0.669	50 of 95	3 of 5
PPO, 1 epoch	1.49	0.123	0.033	0.007	77 of 95	5 of 5

- **A2C stays at the corners**, at every late checkpoint and in all five seeds.
- **One-epoch PPO settles.** It ends with 5 of 5 seeds at the corners, though it takes longer than A2C (mean distance 0.12 after 300 episodes against 0.011).
- **Default PPO does not settle.** It ends with 3 of 5 seeds at the corners and 50 of 95 late checkpoints. Two seeds leave the corners and do not return within 900 episodes (their distance is about 1.0 and over 2 at the end), so the instability is not just a slow start.
- The two PPO settings with a learning rate of 0.003 were not run longer.

A different step length or discount. A2C on the walking game with the maximum step at 0.1 or 0.4, or the discount at 0.5 or 0.99, five seeds each, 300 episodes, next to the default (0.2 and 0.9):

Setting	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
max step 0.1, discount 0.9	1.08	0.439	0.386	0.234	0.184	20 of 35	3 of 5
max step 0.2, discount 0.9 (default)	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5
max step 0.4, discount 0.9	1.67	0.509	0.182	0.036	0.028	32 of 35	5 of 5
max step 0.2, discount 0.5	1.49	0.409	0.088	0.009	0.003	35 of 35	5 of 5
max step 0.2, discount 0.99	1.49	0.836	0.442	0.102	0.056	22 of 35	4 of 5

- **A lower discount makes learning faster and a higher one slower.** A discount of 0.5 gets 5 of 5 seeds to the corners and 35 of 35 late checkpoints, and 0.99 gets 4 of 5 and 22 of 35.
- **A longer step costs nothing and a shorter one is slower.** A step of 0.4 is as reliable as the default (5 of 5, 32 of 35). A step of 0.1 gets 3 of 5 seeds and 20 of 35 late checkpoints within 300 episodes; more episodes might close the gap, which was not run.
- **These change how fast the players get to the corners, not where.** The one-shot Nash corners do not depend on the step length or the discount.
- A2C only, one house setup and five seeds, so these are patterns and not a ranking.

5.13 Other house setups and dog inertia on the walking game

The walking-game runs above use one house setup. Here the three algorithms are run on three others, and A2C with dog inertia (see the walking-game report, Section 4.7, for the discussion):

House setup	Algorithm	Before training	At 100	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
default: (0.88, 0.12) / (0.3, 0.65), w = 0.5	REINFORCE	1.49	0.341	0.055	24 of 35	4 of 5
default: (0.88, 0.12) / (0.3, 0.65), w = 0.5	A2C	1.49	0.199	0.011	34 of 35	5 of 5
default: (0.88, 0.12) / (0.3, 0.65), w = 0.5	PPO	1.49	0.127	0.371	14 of 35	2 of 5
(0.9, 0.2) / (0.1, 0.8), w = 0.5	REINFORCE	1.41	0.569	0.168	14 of 35	2 of 5
(0.9, 0.2) / (0.1, 0.8), w = 0.5	A2C	1.41	0.275	0.070	22 of 35	3 of 5
(0.9, 0.2) / (0.1, 0.8), w = 0.5	PPO	1.41	0.836	0.515	14 of 35	2 of 5
(0.8, 0.8) / (0.2, 0.3), w = 0.3	REINFORCE	1.31	0.707	0.239	18 of 35	4 of 5
(0.8, 0.8) / (0.2, 0.3), w = 0.3	A2C	1.31	0.429	0.036	29 of 35	5 of 5
(0.8, 0.8) / (0.2, 0.3), w = 0.3	PPO	1.31	0.974	0.932	9 of 35	1 of 5

House setup	Algorithm	Before training	At 100	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
(0.7, 0.1) / (0.6, 0.9), w = 0.7	REINFORCE	1.55	0.897	0.848	2 of 35	0 of 5
(0.7, 0.1) / (0.6, 0.9), w = 0.7	A2C	1.55	0.842	0.688	4 of 35	0 of 5
(0.7, 0.1) / (0.6, 0.9), w = 0.7	PPO	1.55	0.838	1.153	4 of 35	1 of 5

Setting	Before training	At 50	At 100	At 200	At 300	Late checkpoints at the corners (from 150)	Seeds at the corners at the end
w0 = 0 (default)	1.49	0.731	0.199	0.032	0.011	34 of 35	5 of 5
w0 = 0.3	1.50	0.880	0.459	0.088	0.015	31 of 35	5 of 5
w0 = 0.6	1.51	0.995	0.513	0.187	0.114	25 of 35	4 of 5

- **A2C is the most reliable wherever learning works, and PPO the least or tied for least.** PPO's early lead on the default setup does not carry over to the others.
- **One setup is much harder:** on (0.7, 0.1) / (0.6, 0.9) with w = 0.7 no algorithm gets more than 1 of 5 seeds to the corners in 300 episodes. More episodes were not tried.
- **Dog inertia slows A2C** (5 and 4 of 5 seeds at the corners at `w0` of 0.3 and 0.6, against 5 of 5 without it).

5.14 Is walking to the corners an equilibrium of the walking game?

The walking game is judged against the one-shot Nash corners, so this checks them against the walking game itself (`doggame/walking_equilibrium.py`; see the walking-game report, Section 4.8). If one player walks straight to its corner and stays, the other's best response is a deterministic optimal-control problem, solved numerically:

House setup	Players start	Red's gain	Blue's gain	Largest gain as a share of the reward
default: (0.88, 0.12) / (0.3, 0.65), w = 0.5	on their houses	0.00000	0.00173	0.29%
default: (0.88, 0.12) / (0.3, 0.65), w = 0.5	on the corners	0.00000	0.00000	0.00%
(0.9, 0.2) / (0.1, 0.8), w = 0.5	on their houses	0.00248	0.00248	0.13%
(0.9, 0.2) / (0.1, 0.8), w = 0.5	on the corners	0.00000	0.00000	0.00%
(0.8, 0.8) / (0.2, 0.3), w = 0.3	on their houses	0.00004	0.00297	3.09%
(0.8, 0.8) / (0.2, 0.3), w = 0.3	on the corners	0.00000	0.00000	0.00%
(0.7, 0.1) / (0.6, 0.9), w = 0.7	on their houses	0.01524	0.03014	4.74%
(0.7, 0.1) / (0.6, 0.9), w = 0.7	on the corners	0.00000	0.00000	0.00%

- **Resting on the corners is an exact equilibrium** in all four setups (gain 0).
- **Walking straight to them is a near-equilibrium:** the best deviation gains 0.13% to 4.7% of the reward, because each step's best position lies outside the square. The full game's equilibrium with feedback strategies was not computed.

5.15 Does the one-shot profile survive dog inertia?

For each of the five main-game house setups, I checked the one-shot Nash actions against unilateral open-loop deviations with the opponent fixed at its Nash action. In this separable squared-distance game, a player's one-step dog target lies in a coordinate-wise interval determined by its action box and the opponent's fixed action. In all five setups, every coordinate satisfies one of two sufficient conditions: (a) the house lies beyond the interval and the initial dog lies on the same side, so the nearest endpoint is the best next dog state and inertia preserves that side; or (b) the dog starts at the house, where following earns the maximum possible reward, zero. Induction therefore proves that repeating the one-shot action maximizes each period's reward against every unilateral action sequence, for any finite or discounted infinite horizon and every `w0` in [0, 1]. The code checks these coordinate conditions in `certifies_open_loop_best_response` and uses a convex optimizer when the certificate does not apply.

The certificate covers all five house setups and both players (10 profile checks). The 15 saved rows at `w0` of 0.3, 0.6 and 0.9 report zero unilateral gain. A negative control with an opponent action changed to (0, 0) fails the certificate and the optimizer finds red a 0.214 gain. The earlier midpoint negative control was invalid for this setup and has been corrected.

- **Scope:** this proves the one-shot profile is a best response to unilateral action sequences while the other player holds its action fixed, from the stated initial states, for any horizon and `w0` in [0, 1].

The feedback equilibrium. A Markov-perfect equilibrium must also be optimal from dog positions that play never reaches. Rewards are sums over the two coordinates and the dynamics and action boxes act coordinate by coordinate, so the game splits into two 1D games; each is solved exactly by Nash value iteration on 401 dog positions and 101 actions per coordinate (`doggame/inertia_dp.py`), and cross-checked state by state with the one-shot deviation principle, using the closed-form value of holding the Nash actions (`doggame/inertia_mpe.py`). The table gives the largest action gap between the two on the path from the starting dog position, and the share of dog positions where they differ:

House setup	w0 = 0.3: path gap / states differing (x, y)	w0 = 0.6: path gap / states differing (x, y)	w0 = 0.9: path gap / states differing (x, y)
(0.9, 0.2) / (0.1, 0.8), w = 0.5	0.00 / 0%, 0%	0.00 / 0%, 0%	0.00 / 0%, 5%
(0.8, 0.8) / (0.2, 0.3), w = 0.3	0.00 / 0%, 29%	0.00 / 5%, 30%	0.00 / 14%, 30%
(0.7, 0.1) / (0.6, 0.9), w = 0.7	0.00 / 57%, 0%	0.01 / 76%, 0%	0.00 / 84%, 0%
(0.95, 0.5) / (0.05, 0.5), w = 0.5	0.50 / 0%, 100%	0.50 / 0%, 100%	0.50 / 0%, 100%
(0.6, 0.6) / (0.4, 0.4), w = 0.5	0.00 / 15%, 15%	0.00 / 51%, 51%	0.00 / 68%, 68%

- **On the path, the feedback equilibrium is the one-shot Nash profile** in every setup and inertia (the 0.01 gap is one action-grid step). In the shared-y setup the gap of 0.5 is not a disagreement: the dynamic program picks another member of that setup's continuum of equilibria.
- **Off the path it is not.** When the dog already sits past a player's house in its own direction, holding the extreme action overshoots, and the player eases off. Holding the one-shot actions at every state is Markov-perfect only in setup 1 at `w0` up to 0.6.
- **Section 5.20 explains these numbers in closed form**, including exactly where holding fails and what the players do there.
- **Limits.** The answer is on a grid with linear interpolation, starting best-response iteration from the corners; one setting (setup 2, `w0 = 0.6`, the y coordinate) has 56 states with no pure grid equilibrium. Uniqueness of the feedback equilibrium is not established.

5.16 Larger PPO network, dog inertia and longer training on the walking game

These exploratory additions use five seeds and the existing walking score (10 sampled games per checkpoint; corner threshold 0.1). A 64-unit PPO network with one epoch ended under the threshold in 4/5 seeds (mean distance 0.218; 26/35 late checkpoints); with four epochs it did so in 1/5 (mean 0.704; 7/35). The 16-unit default with four epochs did so in 2/5 (mean 0.371; 14/35). PPO with `w0 = 0.3` ended under threshold in 4/5 seeds (mean distance 0.200; 24/35 late checkpoints), while `w0 = 0.6` did so in 2/5 (mean 0.569; 9/35). These five-seed samples are noisy; the network result is not a stable algorithm ranking. No entropy bonus was tested.

On the hardest walking setup, the 900-episode study is now complete (five seeds per algorithm; late checkpoints start at episode 450):

Algorithm	Mean distance at 900	Seeds under 0.1 at 900	Late checkpoints under 0.1
REINFORCE	0.563	2 of 5	29 of 95
A2C	0.103	4 of 5	68 of 95
PPO (4 epochs)	1.249	1 of 5	21 of 95

The standard deviations of the final distances are 0.628, 0.205 and 0.775, respectively. This reinforces the difficulty of the setup but does not settle the algorithm ranking: there are five seeds, and one PPO seed ended exactly at the corners while the other four remained far away.

An entropy bonus. Default PPO (four epochs) on the walking game for 900 episodes, with each player's loss rewarding its policy's entropy (angle and radius; `entropy` in `doggame/train_ac.py`), five seeds per coefficient; coefficient 0 reproduces the default row of Section 5.12 exactly:

Entropy coefficient	Before training	At 300	At 600	At 900	Late checkpoints at the corners (from 450)	Seeds at the corners at the end
0.0	1.49	0.371	0.499	0.669	50 of 95	3 of 5
0.01	1.49	0.078	0.277	0.316	55 of 95	2 of 5
0.03	1.49	0.135	0.072	0.063	66 of 95	3 of 5
0.1	1.49	0.307	0.220	0.271	12 of 95	0 of 5

- **A moderate bonus helps on average but does not make PPO reliable.** At 0.03 the mean distance after 900 episodes falls from 0.67 to 0.06 and the late checkpoints at the corners rise from 50 to 66 of 95, but still only 3 of 5 seeds end at the corners, as without the bonus; a large bonus (0.1) is worse. One-epoch PPO (77 of 95, 5 of 5) and A2C (95 of 95, 5 of 5) remain more reliable, which again points at the reuse of each episode.

A tighter clipping range. If four epochs on one 15-step episode move the policy too far, limiting each update's step should help. PPO with four epochs and a tighter clip, 900 episodes, five seeds per setting:

PPO setting	Before training	At 300	At 600	At 900	Late checkpoints at the corners (from 450)	Seeds at the corners at the end
4 epochs, clip 0.2 (default)	1.49	0.371	0.499	0.669	50 of 95	3 of 5
4 epochs, clip 0.1	1.49	0.416	0.326	0.372	60 of 95	3 of 5
4 epochs, clip 0.05	1.49	0.217	0.160	0.046	80 of 95	4 of 5
1 epoch, clip 0.2	1.49	0.123	0.033	0.007	77 of 95	5 of 5

- **Tighter clipping steadies PPO step by step.** With clip 0.05 the mean distance after 900 episodes is 0.046 (0.669 at the default 0.2), 80 of 95 late checkpoints are at the corners and 4 of 5 seeds end there, close to one-epoch PPO. Together with the epoch and learning-rate results, this supports the explanation that PPO is unstable here because reusing each short episode for several epochs moves the policy too far per update; it rests on five seeds per setting.

5.17 Relation to prior work and claims boundary

The static, no-inertia dog game is a bounded, two-player instance of the affine-receiver squared-loss structure in the Battling Influencers Game (BIG). BIG already proves the potential-game structure and characterizes pure equilibria; this repository does not claim that basic equilibrium structure as new. The repo's contribution is an executable empirical and verification study of policy-gradient and Nash-Q methods on this tractable game, with explicitly modeled inertia and movement-constrained variants. These are exploratory extensions, not evidence of a new general algorithm or a top-conference contribution. The dynamic results (the feedback equilibrium with inertia and the Markov-potential counterexample) are, as far as the literature checked here goes, not reported for this game, but they are instances of known phenomena rather than new theory. See the references in Section 9.

5.18 Absolute polar actions on the original dog game

This study implements the policy-gradient part of the handwritten notes. The player actor samples a von Mises angle and a Beta radial fraction from the square's center, then maps that pair to an absolute feasible point using the direction-dependent ray length. The original game's dog location is the state; houses stay fixed for a run. DQN/Q-value sketches and the separate pursuit-game sketch were excluded as requested. The user confirmed the polar origin; remaining choices are in ASSUMPTIONS.md.

The primary comparison was fixed at the first house setup, with ten seeds for each representation and REINFORCE, A2C and PPO. It compares the clipped Cartesian Gaussian, a bounded Cartesian Beta control and the polar Beta/von-Mises package. All methods used 1,500 episodes, 20 steps, learning rate 0.01 and separate player networks; PPO used four epochs and clip 0.2. At episode 1,500, the maximum one-shot unilateral expected-payoff gain was computed from 8,192 sampled actions at the reset state. The action-moment payoff was calculated exactly, and the best response was projected onto the square. This is not a score over the full state distribution or a full Markov-game equilibrium.

Algorithm	Clipped Gaussian mean (SD)	Cartesian Beta mean (SD)	Polar mean (SD)	Polar - Cartesian Beta, paired 95% t interval
REINFORCE	0.0193 (0.0023)	0.0281 (0.0007)	0.0689 (0.0060)	0.0407 [0.0367, 0.0447]
A2C	0.0205 (0.0020)	0.0269 (0.0015)	0.0620 (0.0051)	0.0350 [0.0317, 0.0383]
PPO	0.0093 (0.0044)	0.0184 (0.0023)	0.1374 (0.1350)	0.1190 [0.0232, 0.2148]

Positive differences favor Cartesian Beta. Every primary polar-vs-Beta paired seed had a larger expected gain (30/30 across the three algorithms); the corresponding unadjusted intervals are exploratory. Only 1/10 clipped-Gaussian PPO runs fell below 0.004, and none of the Beta or polar runs did. That legacy cutoff was chosen for deterministic action pairs, so those counts are descriptive. The polar actors also had greater action variance and greater mean-action distance from the analytical corners; the supplied concise policy-gradient update PDF shows both the learning curves and estimated action means.

After the primary run had begun, a separate exploratory follow-up was specified for REINFORCE on the other four predefined house setups, ten seeds each, comparing only the two exact-support Beta policies. It is reported separately and is not part of the primary protocol:

Red / blue houses; w	Cartesian Beta mean	Polar mean	Polar - Cartesian Beta, paired 95% t interval	Polar lower in pairs
(0.8, 0.8) / (0.2, 0.3); 0.3	0.0340	0.1218	0.0879 [0.0290, 0.1467]	0/10
(0.7, 0.1) / (0.6, 0.9); 0.7	0.0238	0.0216	-0.0022 [-0.0065, 0.0020]	7/10
(0.95, 0.5) / (0.05, 0.5); 0.5	0.0219	0.0184	-0.0035 [-0.0097, 0.0027]	6/10
(0.6, 0.6) / (0.4, 0.4); 0.5	0.0065	0.0151	0.0086 [0.0074, 0.0097]	0/10

The effect varies with the house setup: two intervals favor Cartesian Beta and two include zero; none shows a clear polar advantage. The follow-up intervals are unadjusted for multiple comparisons and omit Monte Carlo scoring error. This study compares policy packages, not a pure coordinate-system ablation: the Gaussian retains clipping and its existing density handling, and network output sizes differ. It is a useful negative/mixed result, not evidence of novelty or publication readiness. See the fixed primary protocol, separate robustness protocol, and raw completed runs in [`results/polar_action_pg.jsonl`](results/polar_action_pg.jsonl) and [`results/polar_action_pg_robustness.jsonl`](results/polar_action_pg_robustness.jsonl).

5.19 The potential structure, equilibrium selection and the Markov-potential test

The one-shot game is an exact potential game. With `d = w a_red + (1 - w) a_blue`, the function `Phi = -||d||^2 + 2 w h_red . a_red + 2 (1 - w) h_blue . a_blue` changes by exactly the deviating player's reward change (`doggame/potential.py`, tested on random deviations in all five setups). This is already proved for the Battling Influencers Game (Wu et al., 2025); it is verified here, not claimed as new. It explains two earlier findings. Moving both players so the dog stays put changes `Phi` by `2 w (1 - w) (h_red - h_blue)` per unit, so `Phi` rises toward the edge wherever the houses differ (boundary equilibria), and is exactly flat in a coordinate the houses share (the continuum of equilibria in the shared-y setup, where the interior point and both mirror boundary points have the same potential).

Equilibrium selection. Because `Phi` is flat along that continuum, the game's structure cannot pick one equilibrium. REINFORCE, separate layout, shared-y setup, 50 seeds per initial exploration noise (seeds 0-9 at the default reproduce Section 5.3 exactly); the last column is a two-sided binomial test that the two mirror images are equally likely, as the game's symmetry (swap the players and reflect x) says they should be:

Initial noise (std)	red top, blue bottom	red bottom, blue top	both at y = 0.5	other	p (mirror images equally likely)
0.14	19 of 50	22 of 50	0 of 50	9 of 50	0.76
0.37	31 of 50	19 of 50	0 of 50	0 of 50	0.12
0.61 (default)	31 of 50	19 of 50	0 of 50	0 of 50	0.12
1.00	30 of 50	20 of 50	0 of 50	0 of 50	0.20

- **Which mirror image wins looks like a fair coin** at every noise level (p from 0.12 to 0.76). An earlier version of this section, with 10 seeds per level, reported that the noise flips the split (p = 0.0014); with 50 seeds that does not hold up, and the 3-to-1 split of Section 5.3 was a small-sample effect of reusing seeds 0-9.
- **No run ends at the interior point**, at any noise level.

Why the boundary. Along the flat direction the expected policy gradient is zero but its estimate is noisy, so the players' y-means random-walk with the dog held at 0.5, and nothing pulls them back to the interior. Two ablations (50 seeds each, default noise) test what stops the walk:

Policy mean in y, actions and dog	red top, blue bottom	red bottom, blue top	both at y = 0.5	other	Red's final y: quartiles	Red's y outside [0, 1]
sigmoid, clipped (default)	31 of 50	19 of 50	0 of 50	0 of 50	0.00 / 0.99 / 1.00	0 of 50
sigmoid, not clipped	20 of 50	21 of 50	0 of 50	9 of 50	0.03 / 0.64 / 0.97	0 of 50
linear, not clipped	17 of 50	20 of 50	0 of 50	13 of 50	-2.26 / 0.33 / 1.29	38 of 50

- **Without clipping**, runs still end on a mirror boundary (41 of 50) and never at the interior, so clipping is not needed; the spread of red's final y is U-shaped.
- **With an unbounded linear mean in y as well**, the walk does not stop: red's y-mean ends outside the box in 38 of 50 runs (from -17 to 13) while the dog's y stays near 0.5 (median 0.506). The boundary outcomes of the default policy are this walk stopped where the sigmoid saturates and the gradient vanishes.
- **Clipping adds a push.** Averaged over Gaussian exploration noise with clipped actions, the potential is no longer flat: it is lowest at the interior point and highest at both mirror endpoints, by symmetry equally (`smoothed_potential` in `doggame/potential.py`, checked by Monte Carlo). Clipping makes the boundary more decisive (50 of 50 runs on it, against 41 of 50 without) but does not choose between the mirror images.

Not a Markov potential game. In a Markov potential game every 4-cycle of unilateral policy changes, (red, blue) to (red', blue) to (red', blue') to (red, blue') and back, sums to zero. Values are computed exactly by following the deterministic dog (`doggame/markov_potential.py`):

Inertia w0	Policies that ignore the dog	Policies that react to the dog
0.0	+6.7e-15	+7.90
0.3	+3.1e-15	+7.12
0.6	+1.8e-15	+5.63
0.9	+5.6e-15	+1.85

- **With policies that react to the dog, the cycles do not sum to zero**, at any inertia and even without it: the dog game is not a Markov potential game. A player's action changes the dog, which changes what a reacting opponent does next, so the potential property breaks. This is an instance of a known phenomenon (a Markov game whose every state game is a potential game need not be a Markov potential game; Leonardos et al.).
- **With policies that ignore the dog, every cycle is zero** (up to rounding), so the game is a potential game in open-loop strategies at any inertia, which is the setting in which potential differential games reduce to one optimal-control problem (Kavuncu et al., 2021).
- **Consequence.** The convergence theorem for independent policy gradient in Markov potential games does not explain why policy gradient converges in this repository's runs, whose policies react to the dog.

5.20 A closed form for the feedback equilibrium with inertia

The game splits by coordinate, so take one: dog position `x` in [0, 1], next dog `x' = w0 x + (1 - w0)(w a_red + (1 - w) a_blue)`, stage reward `-(x' - h)^2` for a player with house `h`, discount `g`. Let `(a_red*, a_blue*)` be the one-shot Nash actions, `d* = w a_red* + (1 - w) a_blue*`, and

k = (1 - g w0^2) / (1 - g w0) (at least 1), target T = d* + k (h - d*).

Proposition 1 (when holding is Markov-perfect). Holding the one-shot actions at every dog position is a feedback equilibrium if and only if, in every coordinate, each player whose one-shot action is at the top of the

box has `(h - d*) k >= w0 (1 - d*)`, each player at the bottom has `(d* - h) k >= w0 d*`, and no one-shot action is strictly inside the box (when `w0 > 0`). Writing the player's slack as `s = (h - d*) / (1 - d*)` (or `(d* - h) / d*`), the condition is `s >= rho(w0, g) = w0 (1 - g w0) / (1 - g w0^2)`; `rho` rises from 0 to 1 as `w0` goes from 0 to 1, and falls as `g` rises, so more patient players tolerate more inertia. Where it fails, a player at the top holds only for `x <= d* + (h - d*) k / w0`.

Proof. Under holding, the dog moves geometrically to `d*`, so values are quadratic in closed form. By the one-shot deviation principle (discounted, bounded rewards) holding is an equilibrium iff no player gains from a one-round change at any state. That round's payoff is a concave quadratic in the next dog position `z`, whose slope at the held `z` is a positive multiple of `D(x) = (h - d*)/(1 - g w0) - w0 (x - d*)/(1 - g w0^2)`. A player at the top holds optimally iff `D(x) >= 0`, at the bottom iff `D(x) <= 0`, and strictly inside iff `D(x) = 0`, which (with `h = d*` for an interior action) fails at every `x` other than `d*`. `D` is linear in `x`, so checking the endpoints gives the conditions.

Proposition 2 (an explicit feedback equilibrium). Call a coordinate *straddling* when the houses lie on either side of the dog position the players' edges give (for red's house above blue's, `h_blue <= w <= h_red`, so the one-shot actions are red at the top and blue at the bottom). Then the policy

a_i(x) = clip(a_i* + (T_i - w0 x - (1 - w0) d*) / ((1 - w0) w_i), 0, 1), w_red = w, w_blue = 1 - w,

in which each player aims the next dog position at its own target, as far as its weight allows, with the other at its one-shot action, is a feedback equilibrium for every `w0` in [0, 1) and discount in [0, 1). It equals the one-shot profile on the joint hold interval of Proposition 1 and is piecewise affine in `x`, with at most three pieces per player.

Proof outline (the full proofs of both propositions are in docs/INERTIA_PROOFS.md, PDF). Let `g_i(z) = -(z - h_i)^2 + g V_i(z)` be a player's one-round objective as a function of the next dog position under the policy. On the joint hold interval the dog stays in it, so `g_i` is the concave quadratic of Proposition 1, peaked at `T_i`, which lies in the interval. Outside it, for the player that eases off, `g_i(z) = -(z - h_i)^2 + g g_i(next(z))` with `next(z)` nondecreasing and between the target and `z`; the operator on the right is a contraction that preserves "nonincreasing beyond `T_i`" (and "nondecreasing below it" on the other side), so `g_i` is single-peaked at `T_i`. The best reply over any interval of reachable next positions is then the clipped target, which is the policy; the other player keeps its edge because the next position is on the far side of its own target. The two players' easing-off regions do not overlap. The proofs are mine and have not been refereed.

Checks. Per setup and inertia, the share of dog positions where holding fails by Proposition 1 against the share where the dynamic program's equilibrium differs from the one-shot actions, and the largest action gap between Proposition 2's policy and the dynamic program (x, then y):

House setup	w0 = 0.3: hold fails, theory / DP; gap to DP	w0 = 0.6: hold fails, theory / DP; gap to DP	w0 = 0.9: hold fails, theory / DP; gap to DP
(0.9, 0.2) / (0.1, 0.8), w = 0.5	x 0% / 0%; 0.00, y 0% / 0%; 0.00	x 0% / 0%; 0.00, y 0% / 0%; 0.00	x 0% / 0%; 0.00, y 5% / 5%; 0.01
(0.8, 0.8) / (0.2, 0.3), w = 0.3	x 0% / 0%; 0.00, y 30% / 29%; 0.01	x 6% / 5%; 0.00, y 30% / 30%; 0.01	x 14% / 14%; 0.01, y 30% / 30%; 0.01
(0.7, 0.1) / (0.6, 0.9), w = 0.7	x 58% / 57%; 0.01, y 0% / 0%; 0.00	x 76% / 76%; 0.01, y 0% / 0%; 0.00	x 84% / 84%; 0.01, y 0% / 0%; 0.00
(0.95, 0.5) / (0.05, 0.5), w = 0.5	x 0% / 0%; 0.00, y 100% / 99%; shared	x 0% / 0%; 0.00, y 100% / 100%; shared	x 0% / 0%; 0.00, y 100% / 100%; shared
(0.6, 0.6) / (0.4, 0.4), w = 0.5	x 16% / 15%; 0.00, y 16% / 15%; 0.00	x 51% / 51%; 0.00, y 51% / 51%; 0.00	x 68% / 68%; 0.00, y 68% / 68%; 0.00

- **Both propositions match the dynamic program**: the shares agree to about one percentage point, and the explicit policy is within one action-grid step (0.01) everywhere. Its largest one-round deviation gain, with continuation values from simulating the dog rather than from a grid, is below 1e-31 over 101 dog positions in every straddling coordinate.
- **The shared-y coordinate is not covered.** Its houses coincide, so both one-shot actions are interior; the dynamic program picks another member of its continuum. Choosing instead each state's equilibrium with the largest total value ("vertex Nash", `selection="vertex"`) changes the actions but no player's value:

Coordinate	Inertia w0	Mean total value: default / vertex	States whose actions differ	Largest value difference
setup 4 y	0.6	-0.0023 / -0.0023	66%	4.7e-19
setup 4 y	0.9	-0.1659 / -0.1659	10%	6.9e-18
setup 3 x	0.6	-0.1112 / -0.1112	0%	0.0e+00
setup 3 x	0.9	-0.3714 / -0.3714	0%	0.0e+00
setup 1 y	0.6	-1.8897 / -1.8897	0%	0.0e+00
setup 1 y	0.9	-2.2996 / -2.2996	0%	0.0e+00

On 200 random one-coordinate games (houses, weight, inertia and discount drawn uniformly; deviation gains over 41 dog positions):

One-shot equilibrium	Games	Explicit policy is an equilibrium (gain < 1e-9)	Largest one-round gain	Holding is Markov-perfect
straddle	77	77 of 77	0.0e+00	30 of 77
interior	123	70 of 123	2.3e-02	0 of 123

- **Proposition 2 holds in every straddling game** (77 of 77), and holding alone is Markov-perfect in 30 of them, matching Proposition 1.
- **It does not extend to interior one-shot actions**: there the policy is an equilibrium in 70 of 123 games and fails in the rest (largest gain 0.023). A player whose one-shot action is interior holds only at `d*` and pushes at an edge elsewhere, so the dog returns faster than under holding and the other player's best target moves: in one such game the dynamic program's red aims at about 0.785 where the policy aims at 0.771. The feedback equilibrium in that case is open.

Beyond Proposition 2: equilibria in target form. Under any stationary profile, a player's one-round objective `-(z - h)^2 + g V(z)` depends only on the next dog position `z`, not on the current one. So whenever it is single-peaked, the player's best reply at every dog position is to aim at the same target, as far as its weight allows. Proposition 2 is the case where the targets have a closed form; in general they solve a two-dimensional fixed point, target = peak of the objective under the current targets, which `target_equilibrium` in `doggame/inertia_theory.py` finds by damped iteration from Proposition 2's targets. On the same 200 random games:

One-shot equilibrium	Games	Proposition 2's policy	Target fixed point	Fixed point: largest gain	Fixed point: iterations (max)
straddle	77	77 of 77	77 of 77	0.0e+00	33
interior	123	70 of 123	122 of 123	1.4e-05	32

- **The target fixed point is a feedback equilibrium in 199 of 200 games**, including 122 of the 123 with an interior one-shot action, where Proposition 2's policy works in only 70. In the example above it moves red's target from 0.771 to the dynamic program's 0.785.

- **The exception** (game 165) keeps a gain of 1.4e-5 after refining the targets beyond the grid; its objective has two local peaks there, which breaks the "clip toward the peak" reply. This is numerical evidence, not a proof; existence and uniqueness of the feedback equilibrium are not shown.
- **Scope.** The propositions are for this game (one dimension per coordinate, quadratic rewards on the next dog position, a box of actions). Uniqueness of the feedback equilibrium is not shown.

5.21 Many houses

With N houses `h_i` and weights `w_i` summing to 1, the dog is at `d = sum_i w_i a_i`, the one-shot game is still an exact potential game, and in each coordinate its equilibrium is a weighted generalized median: players with houses above the dog push to the top, players below push to the bottom, and at most one player, the *pivot*, stops the dog exactly on its own house from inside the box. This is the known Nash outcome of the average voting rule (Renault and Trannoy, 2005); `doggame/many_houses.py` computes it and the tests check it against the potential's maximum. Proposition 1's argument never used the number of players, so it holds verbatim: with inertia, holding the one-shot actions is Markov-perfect if and only if no pivot is inside the box and every player at an edge has slack at least `rho(w0, g)`.

The study draws N houses from a skewed distribution (Beta(2, 5)) with equal weights, 200 draws per N, discount 0.9:

Houses N	One-shot dog: mean (sd); limit 0.339	No pivot inside (limit 0.34)	Largest inertia: mean	N times it, median when no pivot
2	0.368 (0.115)	0.20	0.0399	0.27
4	0.340 (0.090)	0.28	0.0357	0.43
8	0.340 (0.055)	0.34	0.0206	0.27
16	0.337 (0.038)	0.29	0.0111	0.45
32	0.340 (0.028)	0.35	0.0058	0.33
64	0.341 (0.019)	0.32	0.0024	0.32
128	0.339 (0.013)	0.34	0.0012	0.34
256	0.340 (0.010)	0.34	0.0007	0.33
512	0.340 (0.007)	0.28	0.0003	0.43
1024	0.340 (0.005)	0.32	0.0002	0.35

- **The one-shot dog converges to the solution of `d = 1 - F(d)`** (0.339 here), not to the median house (0.264): the mean rule's generalized median, as Renault and Trannoy show.
- **A pivot is inside the box about two times in three**, and then holding is never Markov-perfect at any inertia. The share without a pivot settles near `1 / (1 + f(d))` (0.34), where `f` is the houses' density at the limit dog; this is a heuristic renewal argument, not a proof.
- **Without a pivot, the largest inertia that keeps holding Markov-perfect shrinks like 1/N** (about 0.35/N here), because the houses nearest the dog leave the edge players a slack of order 1/N. So for any fixed inertia, the static equilibrium of a game with many houses is almost never a feedback equilibrium.
- **The feedback equilibrium with many houses has the same target form.** If every player aims the next dog position at its own target, each round is the one-shot many-house game with the targets as houses, so the dog lands at the generalized median of the targets (`next_dog` in `doggame/many_houses.py`). Targets from the fixed point (`target_equilibrium`) on random games with uniform houses and random weights, inertia and discount:

Players	Games	Target form is an equilibrium	with single-peaked objectives	with several peaks	Largest gain	Holding is Markov-perfect
3	20	20 of 20	19 of 19	1 of 1	4.8e-07	4 of 20
5	20	18 of 20	14 of 14	4 of 6	2.6e-04	0 of 20
8	20	17 of 20	9 of 9	8 of 11	5.7e-04	1 of 20

Aiming at the targets is a feedback equilibrium in 55 of 60 games and in every game where all players' one-round objectives are single-peaked (42 of 42), as the argument behind Proposition 2 suggests; all 5 failures have a player whose objective has several peaks, which become more common with more players. Holding the one-shot actions is Markov-perfect in only 5 of the 60.

- **What exact gradient play finds with many houses.** Noise-free gradient play (`play_many` in `doggame/exact_play.py`) with 3 and 5 players, five random games each, inertia 0.6:

Players	Policy	Games	Largest one-round gain: median (range)	Below holding's gain in
3	target-form equilibrium	5	0.0000 (0.0000 to 0.0000)	5 of 5 (by half or more)
3	hold the one-shot actions	5	0.0646 (0.0350 to 0.1022)	-
3	exact play, table	5	0.0006 (0.0003 to 0.0010)	5 of 5 (by half or more)
3	exact play, network	5	0.0350 (0.0150 to 0.0887)	2 of 5 (by half or more)
5	target-form equilibrium	5	0.0000 (0.0000 to 0.0000)	5 of 5 (by half or more)
5	hold the one-shot actions	5	0.0696 (0.0097 to 0.0766)	-
5	exact play, table	5	0.0003 (0.0000 to 0.0008)	5 of 5 (by half or more)
5	exact play, network	5	0.0177 (0.0027 to 0.0696)	2 of 5 (by half or more)

A table of actions comes within about 0.001 of the equilibrium in all 10 games. One small network per player ends exactly at holding's gain in 6 of 10 games and improves on it by half or more in the other 4, but never reaches the equilibrium, as with two players (Section 5.22).

- **Open:** a proof for many houses, a mean-field formulation of the feedback equilibrium, and sampled policy-gradient learners there.

5.22 Does self-play learn the feedback equilibrium?

Section 5.20 gives a feedback equilibrium that differs from the one-shot actions off the equilibrium path. To give learners a chance to find it, each episode here starts from a uniformly random dog position (five seeds per row unless stated). A learned policy's mean actions are scored by their *feedback exploitability*: the largest and the mean gain, over an 11 x 11 grid of dog positions and both players, from changing one round's action (a 21 x 21 grid) and then following the learned policy again, with the dog simulated exactly. Any feedback equilibrium scores 0, whichever one it is, so this does not assume the equilibrium is unique.

Setting	Policy	Runs	Largest one-round gain: median (range)	Mean one-round gain: median
setup 3, w0 = 0.6	Proposition 2	1	0.0000	0.0000
setup 3, w0 = 0.6	hold the one-shot actions	1	0.0756	0.0114
setup 3, w0 = 0.6	REINFORCE	5	0.1537 (0.1156 to 0.6714)	0.0275
setup 3, w0 = 0.6	A2C	5	0.0756 (0.0755 to 0.0759)	0.0115
setup 3, w0 = 0.6	PPO	5	0.0756 (0.0756 to 0.4599)	0.0115
setup 3, w0 = 0.6	REINFORCE, Beta policy	5	0.0900 (0.0855 to 0.1287)	0.0247

Setting	Policy	Runs	Largest one-round gain: median (range)	Mean one-round gain: median
setup 3, w0 = 0.6	A2C, Beta policy	5	0.0854 (0.0813 to 0.0891)	0.0222
setup 3, w0 = 0.6	PPO, Beta policy	5	0.0991 (0.0700 to 0.4077)	0.0323
setup 3, w0 = 0.6	A2C, 15000 episodes	3	0.0756 (0.0756 to 0.0756)	0.0114
setup 3, w0 = 0.6	A2C, Beta policy, 15000 episodes	3	0.0791 (0.0776 to 0.0798)	0.0150
setup 5, w0 = 0.9	Proposition 2	1	0.0000	0.0000
setup 5, w0 = 0.9	hold the one-shot actions	1	0.2084	0.0397
setup 5, w0 = 0.9	REINFORCE	5	0.2014 (0.1506 to 0.3832)	0.0337
setup 5, w0 = 0.9	A2C	5	0.2084 (0.2084 to 0.2084)	0.0397
setup 5, w0 = 0.9	PPO	5	0.2084 (0.2084 to 0.2084)	0.0397

- **A2C and PPO learn to hold the one-shot actions, not a feedback equilibrium.** Their gains equal holding's in 19 of 20 Gaussian runs across the two settings, so the learned policies are the one-shot profile at every dog position: unexploitable on the equilibrium path, exploitable off it, although every episode started off the path.
- **It is robust to the obvious fixes.** Five times longer training (15000 episodes) leaves A2C exactly at holding; a Beta policy, whose shape parameters do not saturate, ends slightly worse than holding (its mean cannot reach the box's edges); and setup 5 at `w0 = 0.9`, with 3.5 times the off-path incentive, gives the same result. REINFORCE ends further away in setup 3 and only slightly below holding's mean gain in setup 5 (0.034 against 0.040).
- **Not noise, and not saturation alone, but shared parameters.** To separate the learning dynamics from the noise of policy-gradient estimates, `doggame/exact_play.py` runs *exact* gradient play on one coordinate: deterministic policies, each player's payoff averaged over 41 starting dog positions and computed by following the dog, and each player ascending its own gradient at the same time. A table of actions (one parameter per grid position) and a small network shared across positions, with holding's largest gain at 0.0756 (setup 3) and 0.1042 (setup 5):

Setting	Policy	Runs	Within 0.05 of Proposition 2	Within 0.05 of holding	Largest one-round gain: median (range)
setup 3 x, w0 = 0.6	table, clamp	1	1 of 1	0 of 1	0.0000
setup 3 x, w0 = 0.6	table, sigmoid	1	1 of 1	0 of 1	0.0007
setup 3 x, w0 = 0.6	network, 16 units	5	0 of 5	1 of 5	0.0479 (0.0479 to 0.0756)
setup 3 x, w0 = 0.6	network, 64 units	5	0 of 5	0 of 5	0.0479 (0.0479 to 0.0479)
setup 5 x, w0 = 0.9	table, clamp	1	1 of 1	0 of 1	0.0000
setup 5 x, w0 = 0.9	table, sigmoid	1	1 of 1	0 of 1	0.0000
setup 5 x, w0 = 0.9	network, 16 units	5	0 of 5	2 of 5	0.1042 (0.1042 to 0.1042)
setup 5 x, w0 = 0.9	network, 64 units	5	0 of 5	1 of 5	0.1042 (0.1042 to 0.1042)

Exact gradient play reaches Proposition 2's equilibrium with a table in every case, and with a shared network in none of 20 runs: the networks end at holding or learn only one player's off-path easing. A plausible mechanism, not tested: a shared network carries "push to the edge" from the many positions where it is right to the few where easing is right, saturating them before their own gradient arrives, which a table cannot do.

- **A second game: soccer.** The same exact gradient play on the 7 x 5 soccer game (`doggame/soccer_exact_play.py`), a zero-sum Markov game with 2,380 states whose equilibrium is pure in every state, scored by the exact duality gap (0 only at a Markov-perfect equilibrium): a table of move logits per state against one 64 x 64 network per player, 2000 Adam steps.

Policy	Runs	Mean gap over states	Largest gap	Largest gap at the start states
table, 2000 steps	1	0.000	0.001	0.000
network, 2000 steps	5	0.255 (0.003 to 0.356)	1.378 (0.727 to 1.378)	0.755 (0.003 to 0.873)

The table reaches the equilibrium (largest gap 0.001); no network does (largest gap at least 0.73 in every run). One network run gets close on most states (mean gap 0.003) but leaves some far off; the other four stall at mean gaps of 0.25 to 0.36. So the table-against-network split is not specific to the dog game.

- **Consequence.** Low exploitability on the equilibrium path, which the earlier sections measure, does not show that a learned policy is a feedback equilibrium of the dynamic game.

5.23 Continuous time and a noisy dog

Shrink the time step: with step `dt`, inertia `w0 = 1 - dt` (the dog moves a fraction `dt` toward the players' average) and discount `exp(-r dt)`, the dog follows `dx/dt = w a_red + (1 - w) a_blue - x` in the limit. Then `k -> (r + 2)/(r + 1)` and `rho -> (r + 1)/(r + 2)` (`continuous_limit` in `doggame/inertia_theory.py`), so:

- **Holding the one-shot actions is a feedback equilibrium in continuous time iff every edge player's slack is at least (1 + r)/(2 + r)**, which is 1/2 for patient players.
- **Proposition 2's equilibrium becomes bang-bang**: the band of dog positions where a player's action is strictly inside the box shrinks in proportion to `dt`, so each player pushes its edge toward its target `d* + (r + 2)/(r + 1) (h - d*)` and switches there. This is a statement about the limit of the discrete-time equilibria; that the limit is an equilibrium of the continuous-time game is not proved here.
- **The box is what makes the cost-free game well posed.** Without it each player's Hamiltonian is linear in its action and unbounded, which is why a Riccati benchmark for this game needs an action cost; with one, linear-quadratic feedback policies are affine and do not depend on the noise (certainty equivalence; Basar and Olsder, 1999).

A noisy dog. Add a Gaussian shock of standard deviation `sigma sqrt(dt)` to each step (the time discretization of `sigma dW`) and solve the dynamic program with Gauss-Hermite expectations (`noise` in `doggame/inertia_dp.py`), on setup 3's x coordinate with `r = 0.5`:

Dog	Time step	Noise sigma	Red switches at	Blue switches at	Value iteration
closed form (Proposition 2)	0.05	0	0.718	0.520	-
closed form (Proposition 2)	0.025	0	0.709	0.527	-
continuous-time limit	0	0	0.700	0.533	-
clipped to [0, 1]	0.05	0.0	0.725	0.520	554 sweeps
clipped to [0, 1]	0.05	0.1	0.760	0.530	550 sweeps
clipped to [0, 1]	0.05	0.2	0.765	0.545	559 sweeps

Dog	Time step	Noise sigma	Red switches at	Blue switches at	Value iteration
clipped to [0, 1]	0.05	0.4	0.790	0.585	did not converge
clipped to [0, 1]	0.025	0.0	0.710	0.530	1107 sweeps
clipped to [0, 1]	0.025	0.2	0.765	0.550	1117 sweeps
unclipped	0.05	0.0	0.725	0.520	554 sweeps
unclipped	0.05	0.02	0.730	0.525	551 sweeps
unclipped	0.05	0.05	0.740	0.525	548 sweeps
unclipped	0.05	0.1	0.760	0.530	550 sweeps
unclipped	0.05	0.2	0.765	0.545	559 sweeps
unclipped	0.05	0.4	0.775	0.585	605 sweeps

- **The equilibrium stays bang-bang, but noise moves the switching points.** Red, whose house is at the one-shot dog, waits longer before easing off (0.725 without noise to 0.765 at `sigma = 0.2`), and blue pushes up over a wider range (0.520 to 0.545), unlike the linear-quadratic benchmark, whose policies ignore the noise.
- **The shift grows smoothly with the noise**, roughly in proportion for small noise (red 0.725, 0.730, 0.740 and 0.760 at `sigma` = 0, 0.02, 0.05 and 0.1 on the unclipped dog), then levels off; it is not a jump at zero noise.
- **It is not the clipping.** With the dog living on [-0.5, 1.5], so that it is never clipped inside [0, 1], the switching points are the same. The shift is also the same with half the time step, and finer state and action grids give the same values (checked separately).
- **Limits.** One setup; the clipped run at `sigma = 0.4` did not converge in 8000 sweeps; why noise moves the switching points this way is not tested. Continuous-time q-learning on this game (the direction of Jia and Zhou, 2023) is open.

5.24 Continuous moves for Nash-Q (the week-7 notes)

The group's week-7 notes ask for a Q-network over continuous actions, Q(s, a_red, a_blue), a fast best response from it, both players moving at once, and paths from interesting starts. `doggame/continuous_q.py` does this on the walking game (houses as in Section 5.9, maximum step 0.2, discount 0.9, 15 steps): a move is any vector in the player's move set, one network reads the state and both players' moves and outputs both Q-values, the learning target is `r + discount * Q_target(s', equilibrium moves)`, and the equilibrium moves of each stage game come from simultaneous gradient play on the network (each player steps up the gradient of its own Q, then projects back onto its move set). Three move encodings answer the notes' "which coordinate system is better?": Cartesian (vx, vy) in a disk, polar (r, theta) in the same disk, given to the network as (r, cos theta, sin theta), and a square ("could do square as well"). Five seeds, 300 episodes; the distance is the players' summed distance from the one-shot Nash corners after 15 greedy steps:

Move encoding	Seeds	From the houses	From random starts	Seeds under 0.1 from random starts
cartesian	5	0.000 (0.000 to 0.001)	0.000 (0.000 to 0.001)	5 of 5
polar	5	0.461 (0.461 to 0.564)	1.061 (0.969 to 1.213)	0 of 5
square	5	0.000 (0.000 to 0.014)	0.000 (0.000 to 0.014)	5 of 5

- **Cartesian moves work and polar moves do not.** Cartesian and square networks bring both players to the corners from the houses and from eight random starts in every seed; polar networks in none.

Fast best responses. Each player's best move against a fixed opponent move on the trained networks, at 50 random states: a dense grid (the reference), gradient ascent by autograd and by finite differences (30 steps, projected), and a quadratic fitted through five evaluations along the two axes, jumping to its vertex and projecting (three rounds with a shrinking spacing):

Encoding	Method	Shortfall in Q	Share of the Q spread	Q evaluations	Milliseconds
square	grid	0.0000	0.0%	1681	0.36
square	autograd	0.0742	14.3%	30	5.71
square	finite	0.0742	14.2%	120	4.41
square	quadratic	0.0237	4.5%	15	0.45
polar	grid	0.0000	0.0%	1722	0.32
polar	autograd	0.3897	27.2%	30	5.76
polar	finite	0.3897	27.2%	120	4.69
polar	quadratic	0.4241	31.7%	15	0.41
cartesian	grid	0.0000	0.0%	1257	0.23
cartesian	autograd	0.0392	10.6%	30	4.00
cartesian	finite	0.0392	10.6%	120	3.14
cartesian	quadratic	0.0426	9.6%	15	0.30

- **A batched grid is exact and, on a CPU, also the fastest**, because its 1,257 to 1,722 evaluations run as one batch. The quadratic fit needs only 15 evaluations and is about as good as gradient ascent or better; gradient ascent leaves 10 to 14% of the Q spread on Cartesian and square moves, so the learned Q has several local peaks over a player's moves. Autograd and finite differences agree, so the analytic gradient (the notes' "??") is not needed.
- **On polar moves every local method fails** (27 to 32% of the spread), which explains why polar learning fails: its learning target comes from the same gradient play.

Both players at once. Simultaneous gradient play on the learned stage games at 30 random states, by step size (in units of the move set's radius):

Encoding	Step size	Settled within 500	Median iterations	Equilibrium gap (mean)
square	0.03	17%	500	0.0058
square	0.1	40%	500	0.0009
square	0.3	43%	500	0.0005
square	1.0	43%	500	0.0006
square	3.0	43%	500	0.0012
polar	0.03	0%	500	0.7640
polar	0.1	0%	500	0.7012
polar	0.3	3%	500	0.6589
polar	1.0	7%	500	0.6551
polar	3.0	0%	500	0.5676
cartesian	0.03	3%	500	0.0017
cartesian	0.1	13%	500	0.0005
cartesian	0.3	27%	500	0.0006

Encoding	Step size	Settled within 500	Median iterations	Equilibrium gap (mean)
cartesian	1.0	27%	500	0.0007
cartesian	3.0	30%	500	0.0006

- **Step sizes of 0.1 to 1 work best.** The moves rarely stop changing by less than 1e-5 within 500 iterations, but where they hover leaves either player at most about 0.001 to gain on Cartesian and square moves; on polar moves the gap stays at 0.57 to 0.76.

Paths. Greedy equilibrium paths of the seed-0 networks from four starts (`docs/continuous_q_trajectories.svg`): with Cartesian and square moves both players head for their corners from every start and, where a straight line would leave the square, slide along the edge, which is what happens at the turn; with polar moves blue barely moves. Five seeds and one house setup; the reading of the notes is in ASSUMPTIONS.md.

6. Research questions and answers

The statuses are the same as in RESEARCH_QUESTIONS.md.

Q1. Can self-play policy gradient converge to a Nash equilibrium in a general-sum game? *Answered for the dog game; open in general.* 74 of 75 REINFORCE runs and 25 of 25 PPO runs converged (Sections 5.1, 5.2). Independent policy gradient can cycle in general-sum games, so this should not be read as a general result. The dog game is favorable: each best response is an affine function of the other player's action, and the bounded domain turns the recursion into a contraction toward the boundary.

Q2. Does the number of episodes depend on the configuration? *Answered.* Yes, and on the algorithm. At 250 episodes REINFORCE had 13 of 25 runs converged overall, but 0 of 5 on setup 3 and 5 of 5 on setup 5. Any episode count needs the setup and algorithm it was measured on.

Q3. Does pre-training help? *Answered for one target.* Section 5.5: it helps when the source has the same equilibrium corners and hurts when it does not.

Q4. Which equilibrium does self-play find? *Answered for the shared-y setup.* Never the interior point, always one of two mirror boundary equilibria, regardless of layout; which one is consistent with a fair coin over 50 seeds (Sections 5.3 and 5.19).

Q5. How precisely must the stage game be solved? *Answered for the dog game.* Section 5.6: the error falls about as 1 over the iterations, and re-solving less often does not matter when the transition is state-independent. **Q5b** (the literature) is answered by a short search: for zero-sum games, error propagation in approximate dynamic programming bounds the effect of an approximate stage-game step, with a factor growing like 1/(1 - g)^2 (Perolat et al., ICML 2015); for general-sum Nash-Q no comparable bound was found, and its convergence needs restrictive conditions (Hu and Wellman, 2003; Bowling, 2000).

Q6. How does resolution trade off against quality? *Answered.* Section 5.7: quality is exact from 5 x 5, cost grows steeply, and the equilibrium is always a single action pair. A function approximator becomes attractive when the action set grows past a few hundred actions, which is where the exact solve already takes seconds to minutes.

Q7. Which network architecture? *Answered for the dog game.* All three converge (24, 25 and 25 of 25), and cross-play between independently trained layouts works where the equilibrium is unique; where the houses share a coordinate it fails exactly for pairs from runs that picked different mirror images (Section 5.3):

Setup	Runs	Own pairs: median exploitability	Cross pairs: median	Cross pairs above 0.004	Cross pairs: largest
setup 1 (unique equilibrium)	15	0.00023	0.00032	0 of 150	0.000
shared y (mirror equilibria)	30	0.00010	0.00019	216 of 600	0.249

Q8. When is the equilibrium unique? *Answered.* When the houses differ in every coordinate; each shared coordinate opens a continuum (Section 2.2, confirmed by Section 5.3).

Q9. How is an equilibrium verified? *Answered.* Exploitability for continuous actions and the indifference check for mixed strategies (Section 4).

Q10. How much non-clean mixing is real? *Answered for the dog game.* None of it: the exact equilibrium is one action pair, and the mixing fictitious play shows is the leftover of its first random pick (Section 5.8).

Q11. What transfers to harder games? *Partly answered by the game with inertia.* On-path convergence and the coin-flip selection along a flat potential carry over; the potential structure (Section 5.19) and learning the off-path equilibrium (Section 5.22) do not, and the state-independent transition that collapses Nash-Q to one stage game is lost. Games with more players or no closed-form equilibrium are untested. The solver hardening and the verification checks are game-agnostic.

Q12. Does inertia change anything besides how long it takes? *Answered.* The steady state on the equilibrium path is unchanged, training slows sharply on the harder setup (Section 5.4), and the feedback equilibrium changes off the path: Propositions 1 and 2 give exactly when the one-shot actions stay Markov-perfect and what the players do instead (Section 5.20), while self-play learns the one-shot actions everywhere (Section 5.22).

Q13-Q15. Q13 (vertex Nash for multi-state games) is answered for the game with inertia: picking each state's equilibrium with the largest total value changes the actions in the shared coordinate but no value, because those equilibria are payoff-equivalent (Section 5.20). Q14 (LLM play and inverse reward recovery) is a separate research thread. Q15 (disk-shaped action spaces) is partly addressed by the walking version's angle and step length.

Q16. Do A2C and PPO beat REINFORCE? *Answered.* On the main game PPO is much faster per episode and A2C is not faster than REINFORCE (Section 5.2). With one epoch, matching REINFORCE on gradient steps, PPO is no faster than REINFORCE, so its speed comes from reusing each episode (Section 5.11, Q24). On the walking game the order changes: A2C is the most reliable and PPO is fast but unstable (Section 5.10, Q26).

Q17-Q20. Q17 (which mirror image, and why never interior) is answered for the shared-y setup: the mirror image is a fair coin, and learning random-walks along the flat direction of the potential until the box stops it, as two ablations show (Section 5.19). Q18 is answered (Section 5.4). Q19 is answered below. Q20 (why mismatched pre-training hurts) is answered: pre-training collapses the exploration noise, and resetting it removes most of the harm (Section 5.5).

Q21. Does re-solving less often change the fixed point? *Answered.* No, in this game (Section 5.6).

Q22. Is the exact equilibrium always one action pair? *Answered.* Yes, in all 25 games (Section 5.7).

Q23. Is the one-shot corner pair an equilibrium of the walking game? *Answered for the resting state and the straight walk.* Resting on the corners is an exact equilibrium, and walking straight to them is a near-equilibrium with gains of 0.13% to 4.7% (Section 5.14). The full feedback equilibrium was not computed.

Q25. How does the walking game change with the step length and the discount? *Partly answered for A2C on one setup.* A lower discount makes learning faster, a higher one slower, a longer step costs nothing and a shorter one is slower within 300 episodes (Section 5.12). The corners themselves do not depend on either.

Q24. Does PPO stay ahead when matched on gradient steps? *Answered for the main game.* No clear lead per step: with one epoch PPO is no faster than REINFORCE, and with 4 epochs it is in line with REINFORCE given the same number of steps (Section 5.11).

Q26. Do A2C and PPO beat REINFORCE on the walking game? *Answered for one house setup (15 runs).* A2C is the most reliable, REINFORCE is slower, and PPO is the fastest but does not stay at the corners (Section 5.10).

Q30. Do the walking-game findings hold on other house setups and with inertia? *Partly answered.* A2C stays the most reliable where learning works, one setup is much harder, and inertia slows learning. New five-seed PPO inertia results extend, but do not resolve, generalization (Sections 5.13 and 5.16). **Q31** is partly answered for network capacity; entropy remains untested.

Q27. Why is PPO unstable on the walking game? *Answered for these settings.* Reusing each short episode for several epochs moves the policy too far per update: fewer epochs and a tighter clipping range both steady it (clip 0.05 brings 4 of 5 seeds to the corners after 900 episodes, against 3 of 5 at the default), while a smaller learning rate and an entropy bonus do not (Section 5.16); five seeds per setting.

Q28-Q29. *Answered for the settings tried.* One-epoch PPO reaches the corners in 4/5 seeds after 300 and 5/5 after 900 episodes, but stays slower than A2C; a smaller learning rate does not help; default PPO does not settle with more training (Sections 5.11 and 5.12). The smaller-learning-rate settings were not run longer.

Q31. Does an entropy bonus or a larger network change PPO's instability? *Answered for these settings: neither makes it reliable.* A 64-unit network helps with one epoch but not with four (Section 5.16), and an entropy bonus of 0.03 lowers the mean distance after 900 episodes (0.67 to 0.06) without raising the number of seeds at the corners (3 of 5); five seeds per setting.

Q32. Does a polar absolute-action policy improve policy-gradient learning on the original dog game? *Partly answered for the tested setups.* On the primary house setup, polar had higher one-shot expected gain than Cartesian Beta in all 30 algorithm-by-seed pairs. The REINFORCE follow-up across four additional setups is mixed: Cartesian Beta is better in two and the intervals include zero in two. The comparison changes distribution family as well as coordinates and is scored only at the reset state (Section 5.18); it does not establish a general polar-policy effect.

Q19. Does the one-shot equilibrium remain an equilibrium with inertia? *Answered.* On the equilibrium path the exact feedback equilibrium is the one-shot Nash profile; off the path it differs. Proposition 1 gives the exact condition for holding the one-shot actions to be Markov-perfect, which holds in setup 1 up to `w0 = 0.6` only (Sections 5.15 and 5.20).

Q33. Is the dog game a Markov potential game? *Answered: no.* 4-cycles with reacting policies do not sum to zero, while open-loop cycles do (Section 5.19).

Q34. Is there an explicit piecewise-affine feedback equilibrium? *Partly answered.* Yes whenever the houses straddle the dog (Proposition 2, with a proof; 77 of 77 random games). In general, equilibria in target form, with targets from a two-dimensional fixed point, work in 199 of 200 random games, including 122 of 123 with an interior one-shot action; this is numerical, and one game keeps a gain of 1.4e-5 (Section 5.20).

Q35. What happens in a noisy, continuous-time dog game? *Partly answered.* In the continuous-time limit Proposition 2's equilibrium becomes bang-bang with explicit switching points, and holding needs slack at least

(1 + r)/(2 + r); a noisy dog keeps the equilibrium bang-bang but moves the switching points, unlike a linear-quadratic benchmark (Section 5.23). Continuous-time q-learning is open.

Q37. Does self-play learn a feedback equilibrium with inertia? *Answered for two settings: no.* From random starts, A2C and PPO learn to hold the one-shot actions everywhere, which is exploitable off the equilibrium path; longer training and a Beta policy do not change this. Noise-free gradient play finds the equilibrium with a table of actions but not with a shared network, so function approximation, not sampling noise, is what stops it (Section 5.22).

Q38. Which move coordinates work for Nash-Q with continuous moves? *Answered for the walking game.* Cartesian (or square) moves: every seed reaches the corners; polar moves: none, because gradient play in polar coordinates does not find the stage-game equilibrium (Section 5.24).

Q39. How should a best response over a continuous move be computed? *Answered for these networks.* A batched grid is exact and fastest on a CPU; a five-point quadratic fit needs 15 evaluations and does as well as gradient ascent, which stops at local peaks (Section 5.24).

Q36. What happens with many houses? *Partly answered.* The one-shot dog converges to the generalized median `d = 1 - F(d)` (known for the average voting rule), and with inertia the static equilibrium stays a feedback equilibrium only without a pivot and below an inertia of order 1/N (Section 5.21). The many-house feedback equilibrium is open.

7. Limits

- **The proofs of Section 5.20 have not been refereed** (docs/INERTIA_PROOFS.md), and Proposition 2 covers straddling coordinates only; they are checked numerically against the dynamic program and an exact one-round deviation test.
- Most results rest on five seeds, and the equilibrium-selection study on ten per layout. Percentages are counts out of those, not estimates with error bars.
- The convergence threshold (0.004) and the pre-training and inertia setups are my choices. Training settings (learning rate 0.01, one episode per update) were not tuned for each algorithm.
- PPO and A2C are compared mainly on episodes. Section 5.11 adds a comparison on gradient steps for the main game. Wall-clock time was not compared.
- Policies trained with inertia are scored against the one-shot equilibrium. The walking game is scored against the one-shot corners. Neither is the equilibrium of its own dynamic game.
- The pre-training study has a single target setup. Most walking studies use one house setup (Section 5.13 adds three), and the step-length, discount and inertia studies use A2C only.
- The walking-game scores use 10 sampled games per checkpoint and five seeds, and the paths are noisy, so small differences between algorithms should not be read as a ranking.
- The polar-action comparison uses 10 seeds on one primary setup, plus a separate 10-seed-per-setup REINFORCE follow-up on four configurations. Its score uses Monte Carlo-estimated action moments at the reset state and does not measure the full state distribution; the paired intervals omit that Monte Carlo scoring error.
- Timings in the resolution table were taken on a heavily loaded machine and show growth, not absolute cost.
- The Gaussian policy clips actions to the square before taking the log-probability, which biases the gradient slightly. Every algorithm shares this.

8. Reproducing

```
python3 -m pytest                          # the test suite
python3 -m doggame.experiments             # the original convergence study
python3 -m doggame.dog_game_study          # five-seed convergence and walking runs
python3 -m doggame.dog_game_studies        # the studies in Section 5 (resumable, shardable)
python3 -m doggame.dog_game_summary        # print every table as Markdown
python3 -m doggame.research_report_pdf     # build research_report.pdf from this file
python3 -m doggame.polar_action_study      # run the predeclared 90-run PG comparison (resumable)
python3 -m doggame.polar_action_robustness # run the 80-run cross-house REINFORCE follow-up
python3 -m doggame.polar_action_report     # build its concise PDF, Markdown, and site figure
```

The studies write one line per finished run, so a stopped run keeps its results and a rerun skips what is done.

The code, results and this report are released under the MIT license.

9. References

Selected related work: Wu et al. (2025), Battling Influencers Game; Hu and Wellman (2003), Nash-Q; Hambly et al. (2023), policy gradients for LQ games; Schulman et al. (2017), PPO; Leonardos et al., Markov potential games; Kavuncu, Yaraneri and Mehr (RSS 2021), Potential iLQR; Benenati and Belgioioso (2025), explicit constrained LQ games; constrained nonlinear dynamic games (piecewise-affine feedback Nash); Monderer and Shapley (1996), potential games; Blume (1993), log-linear learning; Fudenberg and Tirole (1991), *Game Theory* (the one-shot deviation principle); Renault and Trannoy (2005), Protecting minorities through the average voting rule, *Journal of Public Economic Theory* 7, 169-199; Basar and Olsder (1999), *Dynamic Noncooperative Game Theory*; Jia and Zhou (2023), q-learning in continuous time, *JMLR*; Perolat, Scherrer, Piot and Pietquin (2015), approximate dynamic programming for two-player zero-sum Markov games, ICML; Bowling (2000), convergence problems of general-sum multiagent reinforcement learning, ICML. The LQ guarantees do not cover this deterministic clipped setup.